
CSS

BARBIER Jean-Matthieu

2026-06-23

Table des matières

1	CSS	1
1.1	Principe	2
1.2	Exemple : CSS Zen Garden	2
1.3	Insérer du CSS dans une page	3
1.3.1	1. Feuille de style externe (recommandé)	3
1.3.2	2. Feuille de style interne	4
1.3.3	3. Style en ligne (<i>inline</i>)	4
1.4	Syntaxe et sélecteurs	4
1.4.1	Syntaxe	4
1.4.2	Sélecteurs	5
1.4.3	Propriétés	8
1.5	Exercices	10
2	Cascade et héritage	12
2.1	Cascade	12
2.1.1	En pratique	12
2.1.2	Correction	13
2.2	Héritage	14
2.2.1	Un sélecteur précis n'empêche pas l'héritage	14
2.2.2	Forcer ou bloquer l'héritage	14
3	Flux et positionnement	15
3.1	Le modèle de boîte	15
3.1.1	Dimensions : <code>width</code> et <code>height</code>	16
3.1.2	Margin, border et padding	16
3.1.3	<code>box-sizing</code> : ce que compte la largeur	17
3.2	Le flux normal	17
3.3	<code>display</code> : le type de boîte	17
3.4	<code>position</code> : positionner une boîte	18
3.5	<code>float</code> et <code>clear</code>	18
3.6	Débordement et visibilité	18
3.7	Exercices	19
4	Responsive, Flexbox, Grid	20
4.1	Le responsive design	20
4.2	Les media queries	21
4.3	Flexbox	21
4.4	Grid	22

1 CSS

admonish note title="Définition" **CSS** = **Cascading Style Sheets** = Feuilles de style en cascade : séquence d'instructions de rendu visuel,

auditif ou tactile d'une ressource HTML ou XML

1.1 Principe

Le CSS est un langage informatique qui permet de décrire la présentation d'un document écrit en HTML ou XML (y compris les dialectes XML comme SVG, MathML ou XHTML).

Enjeux :

- **séparation entre la structure & la présentation**
- présentation selon le media (print / screen / tv...)
- cascade des styles : combinaison de différentes sources de style

1.2 Exemple : CSS Zen Garden

Le projet CSS Zen Garden a été lancé en 2003 par Dave Shea. Il s'agit d'un projet qui met en avant la puissance du CSS en permettant aux designers de créer des mises en page uniques et créatives en utilisant le même contenu HTML. Le site présente une série de designs réalisés par des designers du monde entier, **tous basés sur le même fichier HTML**.

Lien : CSS Zen Garden

La page HTML sans aucune feuille de style :

CSS Zen Garden

The Beauty of CSS Design

A demonstration of what can be accomplished through CSS-based design. Select any style sheet from the list to load it into this page.

Download the example [html file](#) and [css file](#)

The Road to Enlightenment

Littering a dark and dreary road lay the past relics of browser-specific tags, incompatible DOMs, broken CSS support, and abandoned browsers.

We must clear the mind of the past. Web enlightenment has been achieved thanks to the tireless efforts of folk like the W3C, WaSP, and the major browser creators.

The CSS Zen Garden invites you to relax and meditate on the important lessons of the masters. Begin to see with clarity. Learn to use the time-honored techniques in new and invigorating fashion. Become one with the web.

So What is This About?

There is a continuing need to show the power of CSS. The Zen Garden aims to excite, inspire, and encourage participation. To begin, view some of the existing designs in the list. Clicking on any one will load the style sheet into this very page. The HTML remains the same, the only thing that has changed is the external CSS file. Yes, really.

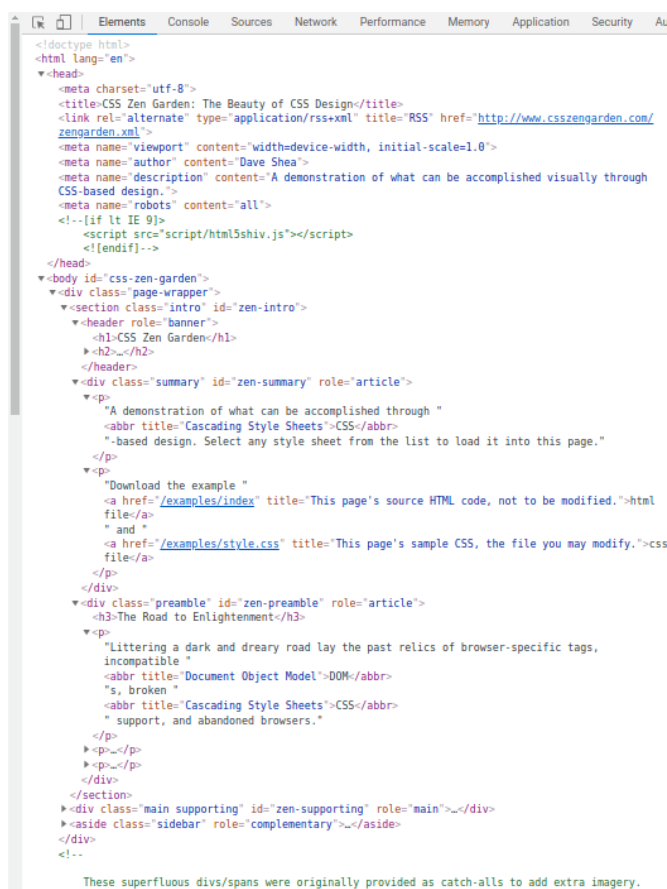
CSS allows complete and total control over the style of a hypertext document. The only way this can be illustrated in a way that gets people excited is by demonstrating what it can truly be, once the reins are placed in the hands of those able to create beauty from structure. Designers and coders alike have contributed to the beauty of the web; we can always push it further.

Participation

Strong visual design has always been our focus. You are modifying this page, so strong CSS skills are necessary too, but the example files are commented well enough that even CSS novices can use them as starting points. Please see the [CSS Resource Guide](#) for advanced tutorials and tips on working with CSS.

You may modify the style sheet in any way you wish, but not the HTML. This may seem daunting at first if you've never worked this way before, but follow the listed links to learn more, and use the sample files as a guide.

Download the sample [HTML](#) and [CSS](#) to work on a copy locally. Once you have completed your masterpiece (and please, don't submit half-finished work) upload your CSS file to a web server under your control. [Send us a link](#) to a archive of that file and all associated assets, and if we



```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>CSS Zen Garden: The Beauty of CSS Design</title>
    <link rel="alternate" type="application/rss+xml" title="RSS" href="http://www.csszengarden.com/zengarden.xml">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="author" content="Dave Shea">
    <meta name="description" content="A demonstration of what can be accomplished visually through CSS-based design.">
    <meta name="robots" content="all">
    <!--[if lt IE 9]>
      <script src="script/html5shiv.js"></script>
    <![endif]-->
  </head>
  <body id="css-zen-garden">
    <div class="page-wrapper">
      <section class="intro" id="zen-intro">
        <header role="banner">
          <h1>CSS Zen Garden</h1>
          <h2></h2>
        </header>
        <div class="summary" id="zen-summary" role="article">
          <p>
            "A demonstration of what can be accomplished through "
            <abbr title="Cascading Style Sheets">CSS</abbr>
            "-based design. Select any style sheet from the list to load it into this page."
          </p>
          <p>
            "Download the example "
            <a href="/examples/index" title="This page's source HTML code, not to be modified.">html file</a>
            " and "
            <a href="/examples/style.css" title="This page's sample CSS, the file you may modify.">css file</a>
          </p>
        </div>
      </section>
      <div class="preamble" id="zen-preamble" role="article">
        <h3>The Road to Enlightenment</h3>
        <p>
          "Littering a dark and dreary road lay the past relics of browser-specific tags, incompatible "
          <abbr title="Document Object Model">DOM</abbr>
          "s, broken "
          <abbr title="Cascading Style Sheets">CSS</abbr>
          " support, and abandoned browsers."
        </p>
      </div>
      <div class="main supporting" id="zen-supporting" role="main"></div>
      <aside class="sidebar" role="complementary"></aside>
    </div>
  </body>
</html>
```

FIGURE 1 – CSS Zen Garden - sans css

Différentes mises en page réalisées par des designers :



Mid Century Modern
ANDREW LOHMAN, United States



Garments
DAN MALL, United States



Steel
STEFFEN KNOELLER, Germany



Apothecary
TRENT WALTON, United States



Handcrafted
ELLIOT JAY STOCKS, United Kingdom



Fountain Kiss
JEREMY CARLSON, United States



FIGURE 2 – CSS Zen Garden - avec css

1.3 Insérer du CSS dans une page

Il existe **trois façons** d'associer du CSS à un document HTML :

1.3.1 1. Feuille de style externe (recommandé)

Le CSS est écrit dans un **fichier séparé** (par exemple `style.css`), relié à la page depuis le `<head>` avec la balise `<link>` :

```
<head>
  <link rel="stylesheet" href="style.css" />
</head>
```

C'est la méthode à privilégier : un même fichier peut mettre en forme **plusieurs pages**, et le fond (HTML) reste bien séparé de la forme (CSS).

1.3.2 2. Feuille de style interne

Le CSS est écrit dans une balise `<style>`, placée dans le `<head>` de la page. Pratique pour une page unique ou un essai rapide :

```
<head>
  <style>
    h1 {
      color: red;
    }
  </style>
</head>
```

1.3.3 3. Style en ligne (*inline*)

Le CSS est écrit **directement sur la balise**, via l'attribut `style` :

```
<h1 style="color: red;">Titre</h1>
```



Le style *inline* est à **éviter** : il mélange le fond et la forme, n'est pas réutilisable, et est difficile à maintenir. On le réserve à des cas très ponctuels.

1.4 Syntaxe et sélecteurs

Références : MDN Sélecteurs : MDC CSS Selectors

1.4.1 Syntaxe

Le CSS associe des **règles de représentation** à des éléments sélectionnés dans le document HTML. Une règle de représentation est composée d'un ou plusieurs **sélecteurs** et d'une **déclaration** qui elle-même est composée d'une ou plusieurs **propriétés** et de leur **valeur**.

Les propriétés sont tout ce qui peut être appliqué à un élément : couleur, taille, marge, police, etc.

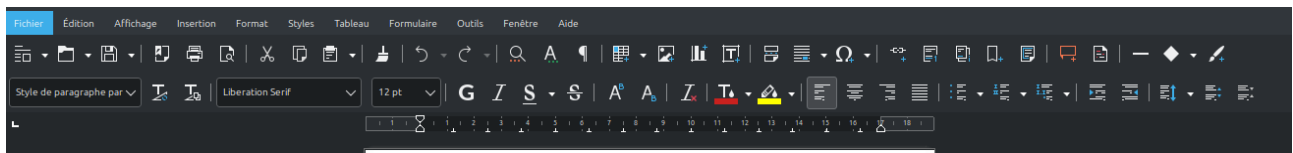


FIGURE 3 – Illustration CSS properties

Exemple de structure :

```
selecteur1 {
  propriete: valeur;
}
```

```
selecteur2,
selecteur3 {
```

```
    propriete: valeur;
}
```

Chaque déclaration se termine par un **point-virgule ;**, et le bloc de déclarations est délimité par des **accolades { }**. On peut placer un même ensemble de propriétés sur **plusieurs sélecteurs** en les séparant par des virgules (selecteur2, selecteur3).

On peut commenter le CSS avec `/* ... */` (il n'existe pas de commentaire sur une seule ligne `//` en CSS) :

```
/* Ceci est un commentaire CSS */
h1 {
    color: red; /* couleur du titre */
}
```

1.4.2 Sélecteurs

Un **sélecteur** désigne les éléments du document auxquels une règle s'applique. On peut cibler les éléments par leur type (la balise), leur position dans l'arbre, leur classe, leur identifiant, leurs attributs, ou leur état (pseudo-classes).

1.4.2.1 Sélecteurs simples, hiérarchie

1. `li` == sélectionne toutes les balises `li` du document
2. `article li` == sélectionne toutes les balises `li` qui sont des descendants (pas forcément directs) d'un `article`
3. `article > ul` == sélectionne tous les `ul` qui sont enfant direct d'un `article`
4. `article > ul > li` == sélectionne tous les enfants direct d'un `ul` descendant direct d'un `article`
5. `h1 + p` == sélectionne toutes les balises `p` qui suivent immédiatement un `h1`
6. `h1 ~ p` == sélectionne toutes les balises `p` qui suivent (immédiatement ou non) un `h1` et qui ont le même parent

Exemple :

```
li {
    background-color: yellow;
}
article li {
    border: 1px solid red;
}
article > ul {
    background-color: blue;
}
article > ul > li {
    border: 3px solid green;
}
h1 + p {
    background-color: orange;
}
h1 ~ p {
    border: 3px solid purple;
}
```

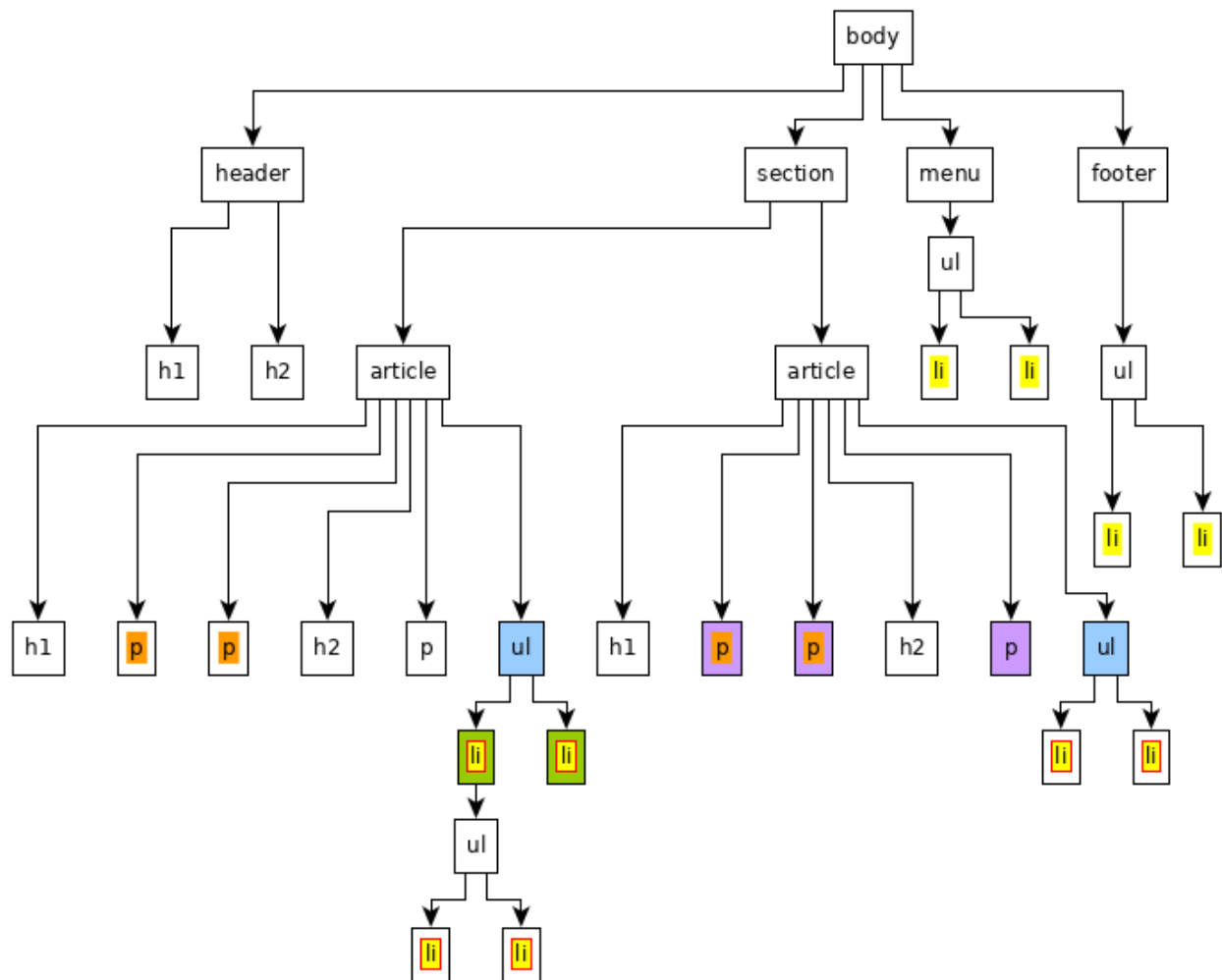


FIGURE 4 – Sélecteur simples & hiérarchie

1.4.2.2 Sélecteur de classe, d'id et d'attribut

- `.rouge` == sélectionne toutes les balises ayant la classe "rouge"
- `h1.rouge` == sélectionne toutes les balises h1 ayant la classe rouge
- `#article-1` == sélectionne toutes (=LA) balise ayant l'id article-1
- `img[alt='Pingouin']` == sélectionne toutes les balises img ayant un attribut alt égal à Pingouin

Plusieurs variantes pour le sélecteur d'attribut : MDN CSS Attribute selectors :

- `attr$=valeur` : sélectionne les éléments dont l'attribut se termine par valeur
- `attr^=valeur` : sélectionne les éléments dont l'attribut commence par valeur
- `attr*=valeur` : sélectionne les éléments dont l'attribut contient valeur
- `attr~=valeur` : sélectionne les éléments dont l'attribut contient valeur
- ...

Exemple :

```

.rouge {
  background-color: red;
}
h1.rouge {

```

```

border: 1px solid red;
}
#article-1 {
background-color: yellow;
}
img[alt="Pingouin"] {
background-color: blue;
}

```

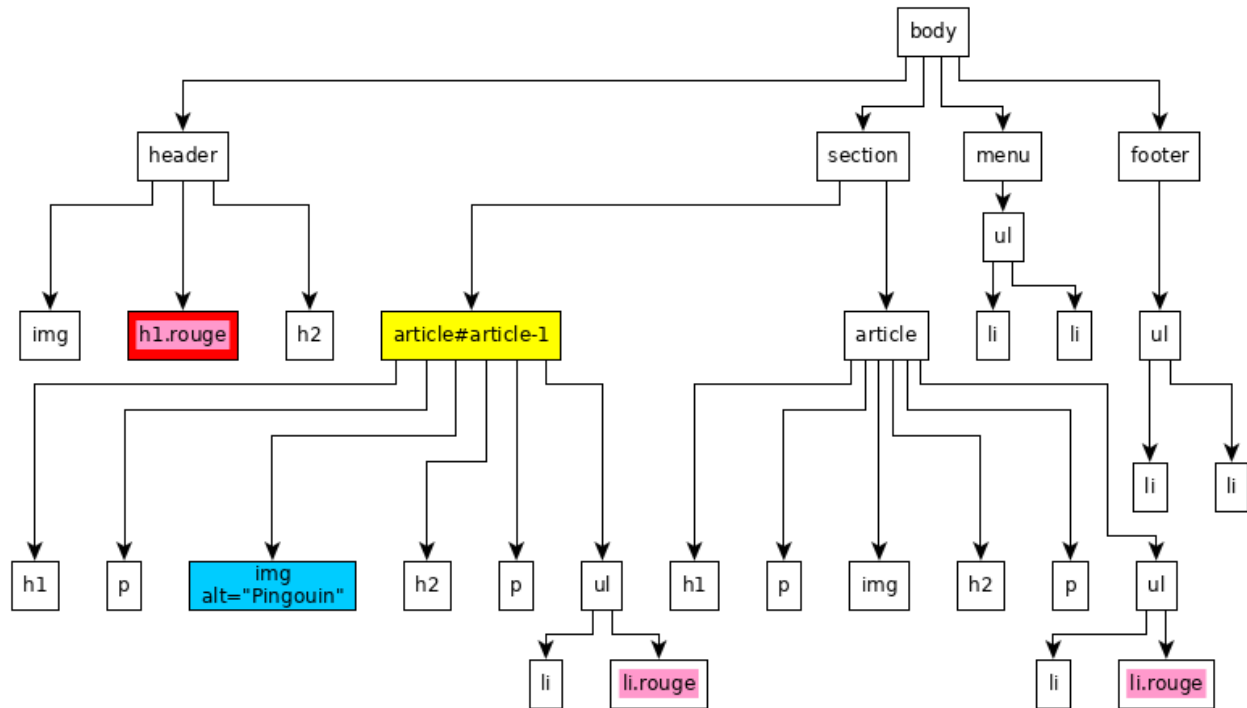


FIGURE 5 – Sélecteurs de classe, d'id et d'attribut

1.4.2.3 Sélecteurs de pseudo-classes et pseudo-éléments

Références : MDN CSS Pseudo-classes et MDN CSS Pseudo-elements

- `:hover` : lorsque la souris survole l'élément
- `:active` : lorsque l'élément est cliqué
- `:focus` : lorsque l'élément a le focus
- `:first-child` : lorsque l'élément est le premier enfant de son parent
- `:last-child` : lorsque l'élément est le dernier enfant de son parent
- `:nth-child(n)` : lorsque l'élément est le n-ième enfant de son parent
- `:nth-of-type(n)` : lorsque l'élément est le n-ième enfant de son parent du même type
- `:before` : avant le contenu de l'élément
- `:after` : après le contenu de l'élément
- `::first-letter` : première lettre de l'élément
- `::first-line` : première ligne de l'élément
- `::selection` : sélection de l'élément
- ...

1.4.3 Propriétés

1.4.3.1 Toutes les propriétés

Les propriétés régissent tous les aspects de la présentation d'un élément : arrière plan, bordures, polices, affichage de texte, positionnement, ...

Liste "antisèche" de 5 pages, très complète, de 2009 (il manque quelques valeurs) et Référence MDN CSS (à jour)

Quelques exemples "classiques" :

- `font-size` : en mm, px, pt, em, rem
- `font-weight` : normal, bold, 200, 300..
- `color` : #RRGGBB ou #RGB ou `rgb(255,255,255)` ou `rgba(255,255,255,1)`
- `background-image` : `url('http://adresse/image')`
- ...

1.4.3.2 Valeurs

Les valeurs dépendent de la propriété; certaines propriétés acceptent des "mots" prédéfinis (ex : `text-align: left|right|start|end|center|justify;`), d'autres acceptent des valeurs dimensionnelles (ex : `font-size: 12px;`), d'autres acceptent des valeurs de type couleur (ex : `color: #RRGGBB;`).

1.4.3.2.1 Valeurs dimensionnelles

- `px` : pixels = pixels "réels" (ex : 12px) sur l'écran
- `pt` : points = 1/72 de pouce (ex : 12pt)
- `mm` : millimètres (ex : 12mm) et `cm` : centimètres (ex : 12cm)
- `em` : taille de la police du **parent**
- `rem` : taille de la police de l'**élément racine**
- `%` : pourcentage de la taille de l'élément parent
- `vh` : 1% de la hauteur de la fenêtre
- `vw` : 1% de la largeur de la fenêtre
- `vm` : 1% de la plus petite dimension de la fenêtre
- `vmax` : 1% de la plus grande dimension de la fenêtre
- ...

1.4.3.2.2 Valeurs de couleur

Référence MDN Color value

- `#RRGGBB` : rouge, vert, bleu (ex : `#FF0000` = rouge)
- `#RGB` : rouge, vert, bleu (ex : `#F00` = rouge)
- `rgb(255,0,0)` : rouge, vert, bleu (ex : `rgb(255,0,0)` = rouge)
- `rgba(255,0,0,1)` : rouge, vert, bleu, alpha (ex : `rgba(255,0,0,1)` = rouge)
- couleurs nommées : red, blue, green, yellow, black, white, ... cf MDN
- ...

1.4.3.2.3 Valeurs calculées

Référence MDN `calc()`

- `calc()` : permet de faire des calculs sur les valeurs dimensionnelles (ex : `width: calc(100% - 20px);` ou `width: calc(100vh - 20px);`)
- `clamp()` : permet de fixer une valeur entre deux bornes (ex : `width: clamp(100px, calc(...), 500px);` ou `width: clamp(100px, calc(...), 500px);`)
- `min()` : permet de prendre la valeur la plus petite (ex : `width: min(100px, 50vw);`)
- `max()` : permet de prendre la valeur la plus grande (ex : `width: max(100px, 50vw);`)

1.4.3.2.4 Variables CSS

Référence MDN CSS Variables

Les variables CSS permettent de définir des valeurs réutilisables dans le document CSS. Elles sont définies avec le préfixe `--` et peuvent être utilisées avec la fonction `var()`. Par exemple :

```
:root {  
  --main-color: #3498db;  
  --secondary-color: #2ecc71;  
}  
h1 {  
  color: var(--main-color);  
}  
p {  
  color: var(--secondary-color);  
}
```

1.5 Exercices



Mettre en forme une page

La page HTML suivante vous est fournie (onglet **HTML**, non modifiable) :

```
<h1>Tux, la mascotte de Linux</h1>
<p>Tux est un manchot, emblème du noyau Linux.</p>
<p class="important">À retenir : Tux est un manchot, pas un pingouin !</p>
<ul>
  <li>Créé en 1996</li>
  <li>Dessiné par Larry Ewing</li>
  <li>Nom proposé par James Hughes</li>
</ul>
<p><a href="https://fr.wikipedia.org/wiki/Tux">Article Wikipédia</a></p>
```

En écrivant **uniquement du CSS** (onglet **CSS**), obtenez la mise en forme suivante :

- le titre <h1> en **bleu** et **centré**
- tous les paragraphes en taille de police **18px**
- les éléments de liste () avec une couleur de fond **jaune clair**
- l'élément ayant la classe important en **gras** et en **rouge**
- le lien (<a>) **sans soulignement**, mais souligné **uniquement au survol** (pseudo-classe :hover)

Rappel : une règle CSS s'écrit `selecteur { propriete: valeur; }`.

HTML :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Tux</title>
</head>
<body>
  <h1>Tux, la mascotte de Linux</h1>
  <p>Tux est un manchot, emblème du noyau Linux.</p>
  <p class="important">À retenir : Tux est un manchot, pas un pingouin !</p>
  <ul>
    <li>Créé en 1996</li>
    <li>Dessiné par Larry Ewing</li>
    <li>Nom proposé par James Hughes</li>
  </ul>
  <p><a href="https://fr.wikipedia.org/wiki/Tux">Article Wikipédia</a></p>
</body>
</html>
```



S'entraîner aux sélecteurs

La page HTML suivante vous est fournie (onglet **HTML**, non modifiable) :

```
<nav class="menu">
  <ul>
    <li><a href="#">Accueil</a></li>
    <li><a href="#">Articles</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
<article>
  <h1>Les manchots</h1>
  <p>Les manchots vivent dans l'<a href="#">hémisphère sud</a>.</p>
  <p>On les classe en plusieurs <a href="#">genres</a> et espèces :</p>
  <ul>
    <li>Aptenodytes
      <ul>
        <li>Empereur</li>
        <li>Royal</li>
      </ul>
    </li>
    <li>Pygoscelis
      <ul>
        <li>Adélie</li>
        <li>Gentoo</li>
      </ul>
    </li>
  </ul>
  <p>Source : <a href="#">Wikipédia</a></p>
</article>
```

La page contient des liens **à deux endroits** (le menu et l'article) et une **liste imbriquée**. En écrivant **uniquement du CSS**, et en réfléchissant au **bon sélecteur** pour chaque cible (sans ajouter de classe dans le HTML) :

- les liens **du menu uniquement** (<a> situés dans .menu) : **sans soulignement** et en **vert** — attention à ne pas styler aussi les liens de l'article !
- les liens **de l'article uniquement** : en **orange**
- le paragraphe qui suit **immédiatement** le <h1> (sélecteur xx + yy) : en **italique**
- **uniquement les genres** (les de **premier niveau** de la liste de l'article), **sans** toucher aux espèces imbriquées : avec une **bordure gauche** (border-left) de 3px verte. Indice : le combinateur xx > yy (enfant direct) permet de cibler sans descendre dans la sous-liste.
- les liens du menu **au survol** (: hover) : en **rouge**

HTML :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Sélecteurs</title>
</head>
<body>
  <nav class="menu">
    <ul>
      <li><a href="#">Accueil</a></li>
      <li><a href="#">Articles</a></li>
      <li><a href="#">Contact</a></li>
```

2 Cascade et héritage

2.1 Cascade

Si plusieurs propriétés sont applicables pour un élément, il faut savoir comment choisir la bonne. La cascade est l'ordre de priorité des propriétés CSS. Il y a trois niveaux de priorité :

1. importance (!important, par exemple `color: red !important`)
2. sélectivité : du plus spécifique au plus général.
 1. la propriété `style` de l'élément
 2. une propriété définie pour l'id d'un élément
 3. une propriété définie par classe ou attribut
 4. une propriété définie pour l'élément
3. ordre d'apparition dans la feuille de style (le plus bas l'emporte)



Calculer la spécificité

La spécificité d'un sélecteur se compte comme un **triplet (id, classe, élément)** : on compte le nombre d'**id**, puis de **classes** (ou attributs, ou pseudo-classes), puis de **balises** qu'il contient. On compare ensuite ces triplets de gauche à droite.

Par exemple :

- `#p1` (un id) vaut (1, 0, 0)
- `.c1` (une classe) vaut (0, 1, 0)
- `div p` (deux balises) vaut (0, 0, 2)
- `p` (une balise) vaut (0, 0, 1)

On lit ces triplets **de gauche à droite**, comme on comparerait des nombres : (1, 0, 0) l'emporte sur tout triplet commençant par 0 (un seul id bat n'importe quel nombre de classes ou de balises). Et (0, 1, 0) l'emporte sur (0, 0, 2) : **une seule classe bat deux balises**, car on compare d'abord la colonne des classes. C'est ce qui explique les cas les plus subtils de l'exemple ci-dessous.

2.1.1 En pratique

Vu le html et le css suivants, quelle sera la couleur de fond de chaque paragraphe ?

```
<p id="p1" class="c1">Texte 1</p>
<div>
  <p class="c1">Texte 2</p>
  <p class="c2" id="p3">Texte 3</p>
  <p class="c3" style="color: yellow;">Texte 4</p>
  <p>Texte 5</p>
</div>
```

```
p { background-color: green; padding: 5px; color: white }
#p1 { background-color: red; }
.c1 { background-color: blue; }
.c2 { background-color: purple; }
```

```
#p3 { background-color: orange; }  
div p { background-color: pink; }  
.c3 { background-color: black; }
```

2.1.2 Correction

Texte 1

Texte 2

Texte 3

Texte 4

Texte 5

FIGURE 6 – Couleurs de fond

Pour chaque paragraphe, c'est la règle `background-color` **la plus spécifique** qui l'emporte :

- **Texte 1** → fond **rouge** : `#p1` (un id, (1, 0, 0)) bat `.c1` et `p`.
- **Texte 2** → fond **bleu** : `.c1` (0, 1, 0) bat `div p` (0, 0, 2) : une classe l'emporte sur deux balises.
- **Texte 3** → fond **orange** : `#p3` (un id) bat `.c2` et `div p`.
- **Texte 4** → fond **noir** : `.c3` bat `div p`; le `style` en ligne ne fixe que la couleur du texte (jaune), pas le fond.
- **Texte 5** → fond **rose** : seule `div p` (0, 0, 2) s'applique, plus spécifique que `p` seul.

2.2 Héritage

Certaines valeurs de propriété appliquées à un élément sont appliquées à ses enfants, d'autres ne le sont pas :

- héritées : `font-family`, `color`, ...
- non héritées : `margin`, `padding`, `border`, ...

En pratique, les propriétés liées au **texte** sont généralement héritées (`color`, `font-family`, `font-size`, `line-height`, `text-align`...), tandis que celles liées à la **boîte** et à la mise en page ne le sont pas (`margin`, `padding`, `border`, `background`, `width`...).



Cascade ≠ héritage

La **cascade** choisit, parmi plusieurs règles qui visent **le même élément**, laquelle l'emporte. L'**héritage** transmet une valeur d'un élément **à ses descendants** lorsqu'aucune règle ne les vise directement.

2.2.1 Un sélecteur précis n'empêche pas l'héritage

Un piège classique apparaît avec les **listes imbriquées**. Soit la règle :

```
article > ul > li {  
  font-weight: bold;  
}
```

Le combinateur `>` est strict : ce sélecteur ne **cible** que les `<li>` de **premier niveau** (enfants directs d'un `<ul>` lui-même enfant direct de l'`<article>`). Les `<li>` d'une **sous-liste** ne sont donc **pas sélectionnés**.

Pourtant, ils s'affichent **aussi en gras**. Pourquoi ? Parce que `font-weight` est une propriété **héritée** : une fois le `<li>` parent en gras, le `<ul>` imbriqué puis ses `<li>` — qui n'ont aucune règle fixant leur `font-weight` — **héritent** la valeur de leur ancêtre. Le sélecteur ne les a pas *choisis* ; ils **héritent** du gras.

Pour limiter le gras au seul premier niveau, on **annule** explicitement l'héritage plus bas :

```
article > ul > li    { font-weight: bold; }  
article > ul > li li { font-weight: normal; } /* coupe l'héritage en profondeur */
```

La même chose se produirait avec `color`, `font-size`... toutes les propriétés héritées. À l'inverse, `border` ou `background` (non héritées) ne « descendent » pas : une bordure posée sur le `<li>` parent n'apparaît pas sur les sous-`<li>`.

2.2.2 Forcer ou bloquer l'héritage

Quelques valeurs spéciales, disponibles pour **toute** propriété, permettent de contrôler l'héritage :

- `inherit` : force la propriété à prendre la **valeur du parent** (même si elle n'est pas héritée par défaut)
- `initial` : remet la propriété à sa **valeur par défaut** (celle définie par le standard CSS)
- `unset` : se comporte comme `inherit` si la propriété est héritable, sinon comme `initial`

```
/* Par défaut, un lien est bleu : on le force à prendre  
   la couleur du texte qui l'entoure */  
a {  
  color: inherit;  
}
```

developer.mozilla.org/en-US/docs/Web/CSS/Reference

admonish note title="Programme" - SNT : thème Web

3 Flux et positionnement

3.1 Le modèle de boîte

Documentation : MDN

En CSS, **chaque élément de la page est représenté par une boîte rectangulaire**. Comprendre comment cette boîte est construite est la base de toute mise en page. Chaque boîte est composée, de l'intérieur vers l'extérieur, de **quatre zones** :

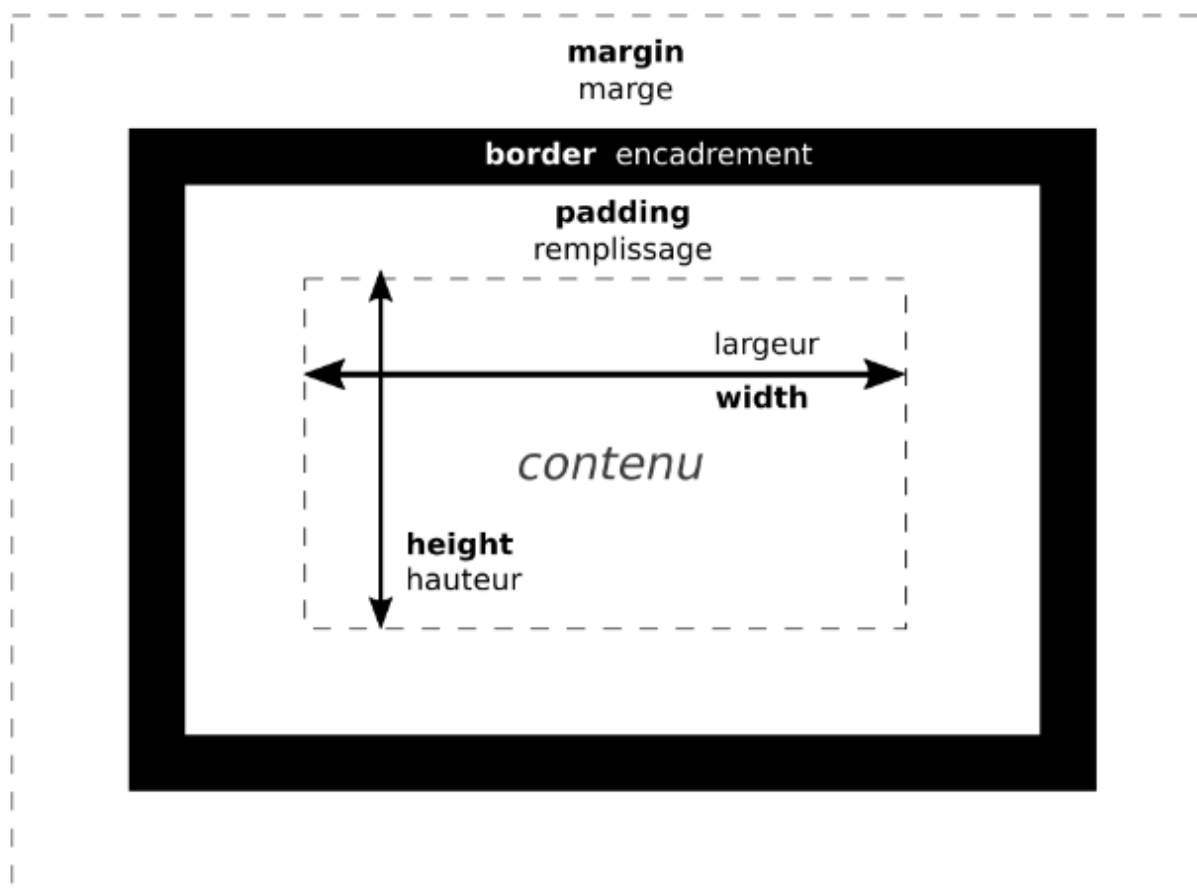


FIGURE 7 – Box model

- le **contenu** (*content*) : le texte, l'image... la zone utile, dont la taille est donnée par `width` et `height`;
- le **padding** : un espace **intérieur**, entre le contenu et la bordure; il prend la couleur de fond de l'élément;
- la **bordure** (*border*) : le trait qui entoure le contenu et son padding;
- la **marge** (*margin*) : un espace **extérieur, transparent**, qui sépare la boîte de ses voisines.



Padding ou margin ?

On confond souvent les deux. Retenez : le **padding** pousse le contenu vers l'intérieur (il agrandit la zone colorée/bordée), tandis que la **margin** éloigne la boîte des autres éléments (espace transparent à l'extérieur).

3.1.1 Dimensions : `width` et `height`

`width` (largeur) et `height` (hauteur) fixent la taille de la **zone de contenu**. On peut aussi borner ces dimensions :

- `min-width` / `max-width` : largeur minimale / maximale
- `min-height` / `max-height` : hauteur minimale / maximale

Les valeurs peuvent être absolues (px) ou relatives (% du parent, vw/vh...).

```
.encadre {
  width: 300px;
  max-width: 100%; /* ne dépasse jamais la largeur du parent */
}
```

Par défaut, un élément **bloc** occupe **toute la largeur** disponible et sa hauteur s'adapte à son contenu; il est donc rare d'avoir à fixer `height`.

3.1.2 Margin, border et padding

`margin`, `padding` et `border` peuvent être réglés **côté par côté** ou d'un seul coup grâce à une notation raccourcie :

- une valeur : les **4 côtés** (`padding: 10px;`)
- deux valeurs : **vertical** puis **horizontal** (`margin: 10px 20px;`)
- quatre valeurs : **haut, droite, bas, gauche** (sens horaire)

On peut aussi cibler un seul côté : `margin-top`, `padding-left`, `border-bottom`...

```
.carte {
  padding: 20px; /* 20px sur les 4 côtés */
  border: 2px solid black; /* épaisseur, style, couleur */
  margin: 10px 0; /* 10px en haut et en bas, 0 à gauche et à droite */
}
```

La bordure se décrit par trois informations : **épaisseur**, **style** (`solid`, `dashed`, `dotted`...) et **couleur**. On peut arrondir ses coins avec `border-radius`.



Centrer un bloc horizontalement

Pour centrer une boîte de largeur fixe dans son parent, on met ses marges gauche et droite à `auto` :

```
.carte {  
  width: 300px;  
  margin: 0 auto;  
}
```

Attention, `margin: auto` ne fonctionne **que sur un élément bloc dont la largeur est fixée**.

3.1.3 box-sizing : ce que compte la largeur

Par défaut (`box-sizing: content-box`), `width` ne mesure **que le contenu** : le padding et la bordure **s'ajoutent** à la largeur visible. Une boîte de `width: 200px` avec `padding: 20px` et `border: 5px` occupe donc en réalité $200 + 2 \times 20 + 2 \times 5 = 250\text{px}$ — souvent déroutant.

Avec `box-sizing: border-box`, `width` inclut le padding et la bordure : la boîte fait **exactement** `200px` de large, quel que soit son padding. C'est beaucoup plus pratique, au point qu'on l'applique souvent à toute la page :

```
* {  
  box-sizing: border-box;  
}
```

3.2 Le flux normal

Sans aucune mise en page particulière, le navigateur place les boîtes selon le **flux normal**, qui dépend du type de chaque élément (notion vue dans la partie HTML) :

- les éléments **bloc** (`<p>`, `<div>`, `<h1>`...) s'empilent **verticalement**, les uns sous les autres, chacun sur toute la largeur disponible;
- les éléments **en ligne** (`<a>`, `<span>`, `<strong>`...) se placent **les uns à la suite des autres horizontalement**, et reviennent à la ligne quand la place manque.

Les propriétés `display`, `position` et `float` servent justement à **modifier ce flux** pour obtenir d'autres dispositions.

3.3 display : le type de boîte

La propriété `display` change la nature de la boîte :

- `block` : bloc (largeur du parent, retour à la ligne avant/après)
- `inline` : en ligne (largeur du contenu; `width/height` **ignorés**)
- `inline-block` : en ligne (reste dans le flux du texte), **mais** on peut lui donner une `width`, une `height`, un padding... — pratique pour aligner plusieurs boîtes côte à côte sans flex
- `none` : l'élément **disparaît** complètement (il n'occupe aucune place, contrairement à `visibility: hidden`)

- `flex`, `grid`, `table` : modes de disposition avancés (vus plus loin)

3.4 position : positionner une boîte

La propriété `position` définit le **référentiel** dans lequel on déplace la boîte à l'aide des décalages `top`, `right`, `bottom`, `left` :

- `static` : position **par défaut** (dans le flux normal ; les décalages n'ont aucun effet)
- `relative` : décalée **par rapport à sa position normale**, tout en gardant sa place dans le flux
- `absolute` : sortie du flux, positionnée **par rapport à l'ancêtre positionné le plus proche** (le premier parent qui n'est pas `static`)
- `fixed` : sortie du flux, positionnée **par rapport à la fenêtre** (reste fixe au défilement)
- `sticky` : reste dans le flux puis **se fige** lorsqu'on atteint un seuil de défilement (entre `relative` et `fixed`)

`z-index` gère la **superposition** des boîtes qui se chevauchent : plus la valeur est élevée, plus l'élément passe **au-dessus**.

3.5 float et clear

Historiquement utilisé pour les mises en page (aujourd'hui remplacé par `flex` et `grid`), `float` sert surtout à **faire contourner un élément par le texte** (par exemple une image entourée de texte) :

- `float: left/right` : l'élément flotte à gauche/droite, le contenu suivant l'enroule ;
- `clear: left/right/both` : force un élément à passer **sous** les éléments flottants.

3.6 Débordement et visibilité

- `overflow` : que faire quand le contenu **déborde** de la boîte ? `visible` (déborde, défaut), `hidden` (coupé), `scroll` (barres de défilement), `auto` (barres si nécessaire) ;
- `visibility` : `visible` ou `hidden` (l'élément devient invisible **mais garde sa place**, contrairement à `display: none`) ;
- `opacity` : opacité de 0 (transparent) à 1 (opaque).

3.7 Exercices



Styliser une carte

Le HTML suivant vous est fourni (onglet **HTML**, non modifiable) :

```
<div class="carte">
  <h2>Tux</h2>
  <p>La mascotte du noyau Linux.</p>
</div>
```

En écrivant **uniquement du CSS**, mettez en forme la carte (sélecteur `.carte`) ainsi :

- une **largeur** de **300px**
- un **padding** de **20px**
- une **bordure** de **2px**, en trait plein (solid), de couleur **gris foncé**
- des **coins arrondis** (border-radius) de **8px**
- une couleur de **fond** gris très clair
- une **marge** de **20px en haut et en bas**, et la carte **centrée horizontalement** dans la page (pensez à `margin: ... auto`)

HTML :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Carte</title>
</head>
<body>
  <div class="carte">
    <h2>Tux</h2>
    <p>La mascotte du noyau Linux.</p>
  </div>
</body>
</html>
```



Des boîtes côte à côte

Le HTML suivant vous est fourni (onglet **HTML**, non modifiable) :

```
<div class="boite rouge">1</div>
<div class="boite verte">2</div>
<div class="boite bleue">3</div>
```

Sans utiliser flex ni grid, et en écrivant **uniquement du CSS** :

- faites en sorte que les trois `.boite` s'affichent **côte à côte** (propriété `display: inline-block`)
- donnez à chaque `.boite` une **largeur** et une **hauteur** de **100px**, un **padding** de **10px**, une **marge** de **5px**, et un texte **centré**
- colorez le fond : `.rouge` en rouge, `.verte` en vert, `.bleue` en bleu
- ajoutez `box-sizing: border-box` aux boîtes et **observez** : la largeur totale reste de 100px alors que le padding est compté à l'intérieur. Essayez de l'enlever pour voir la différence.

HTML :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Boîtes</title>
</head>
<body>
  <div class="boite rouge">1</div>
  <div class="boite verte">2</div>
  <div class="boite bleue">3</div>
</body>
</html>
```

4 Responsive, Flexbox, Grid



Pour information

Cette partie n'est **pas traitée en détail** : elle donne juste un **point d'entrée** à celles et ceux qui voudraient aller plus loin. Chaque section renvoie vers la documentation MDN de référence.

4.1 Le responsive design

Une même page web est consultée sur des écrans très différents : grand moniteur, ordinateur portable, tablette, téléphone... Le **responsive design** (« design adaptatif ») consiste à concevoir une page dont la **mise en page s'adapte automatiquement** à la taille de l'écran : les contenus se réorganisent, les colonnes s'empilent, les images se redimensionnent.

Deux ingrédients de base :

- une page **fluide** (largeurs en %, en vw, ou avec `max-width` plutôt qu'en px fixes);

- la balise `<meta name="viewport">` dans le `<head>`, qui indique au téléphone de ne pas « dé-zoomer » la page :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

En savoir plus : MDN — Responsive design

4.2 Les media queries

Une **media query** applique des règles CSS **seulement si une condition est remplie** — le plus souvent une condition sur la largeur de l'écran. C'est l'outil qui permet de changer la mise en page selon l'appareil.

```
/* Style par défaut (s'applique partout) */
body {
  font-size: 18px;
}

/* Sur les écrans de moins de 600px de large (téléphones) */
@media (max-width: 600px) {
  body {
    font-size: 14px;
  }
}
```

On utilise surtout `max-width` (« jusqu'à telle largeur ») et `min-width` (« à partir de telle largeur »). Les largeurs où la mise en page change s'appellent des **points de rupture** (*breakpoints*).

En savoir plus : MDN — Media queries

4.3 Flexbox

Flexbox est un mode de disposition qui **aligne facilement des éléments sur une ligne (ou une colonne)** et répartit l'espace entre eux. C'est aujourd'hui la méthode la plus simple pour, par exemple, placer des éléments côte à côte ou centrer un contenu.

On l'active en mettant `display: flex` sur le **conteneur** : ses enfants directs deviennent des **éléments flexibles** alignés sur une ligne.

```
.barre {
  display: flex;
  justify-content: space-between; /* répartit l'espace horizontalement */
  align-items: center;           /* centre verticalement */
}

<div class="barre">
  <span>Logo</span>
  <span>Accueil</span>
  <span>Contact</span>
</div>
```

Quelques propriétés clés (sur le **conteneur**) :

- `flex-direction` : `row` (en ligne, défaut) ou `column` (en colonne)

- `justify-content` : alignement **dans le sens principal** (`center`, `space-between`, `space-around...`)
- `align-items` : alignement **dans le sens perpendiculaire** (`center`, `stretch...`)
- `flex-wrap` : `wrap` pour autoriser le passage à la ligne quand la place manque

Combiné aux media queries, Flexbox permet par exemple de passer un menu de l'horizontale (`row`) à la verticale (`column`) sur petit écran.

En savoir plus : [MDN — Flexbox](#)

4.4 Grid

CSS Grid va plus loin que Flexbox : là où Flexbox aligne sur **une** dimension (une ligne *ou* une colonne), Grid organise le contenu sur **deux** dimensions à la fois — en **lignes et colonnes**, comme une grille. C'est l'outil idéal pour des mises en page complètes (en-tête, barre latérale, contenu, pied de page).

```
.grille {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr; /* trois colonnes de largeur égale */  
  gap: 10px; /* espace entre les cases */  
}
```

Le sujet étant vaste, on se contentera ici de ce **survol** et d'un renvoi vers la documentation complète :

En savoir plus : [MDN — Grid](#)